

LLM API Field Guide

An LLM API bills you whether or not the answer was any good, and most of the ways it goes wrong do not raise an exception. Every script here is read only.

16 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 16 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/llm-api-fixes

Guides: allanninal.dev/llm

All 14 repos: github.com/allanninal

The fixes

1 an archived project still holds live API keys

The prototype was shut down in the spring. The project was archived, which felt like closing it: it vanished from the console's project switcher and from every list anyone looks at. Nothing inside it was touched. Two keys are still enabled, one of them authenticated a request last Tuesday, and the quarterly key audit has never seen either of them, because the audit iterates projects and archived projects are not in the default listing.

[openai_archived_project_keys.py](#)

<https://www.allanninal.dev/llm/archived-project-still-holds-keys/>

<https://github.com/allanninal/llm-api-fixes/tree/main/archived-project-still-holds-keys>

2 scheduled jobs pay full price for work the Batch API halves

Nothing here is broken. No request failed, no row is missing, and no alert should have fired. The nightly enrichment job fires forty thousand completions between 02:00 and 02:20, finishes cleanly, and does it again the next night. It has no user waiting on it and no latency requirement of any kind, and it is being billed at the interactive rate because the synchronous endpoint is what the SDK example used. This is not a bug report. It is an invoice roughly twice the size it needs to be.

[openai_batch_discount_audit.py](#)

<https://www.allanninal.dev/llm/batch-discount-left-unused/>

<https://github.com/allanninal/llm-api-fixes/tree/main/batch-discount-left-unused>

3 the batch left an `error_file_id` that nothing ever fetched

The Batch API answers in two files. Successes go to `output_file_id` and failures go to `error_file_id`, and the ingest code was written against the first one on a day when the test batch had no failures. It has never opened the second. The failures are not lost — they were written down carefully, one JSON line each, with the `custom_id` and the reason. They are sitting in a file that has an id, a byte count and an expiry date, and in thirty days they will be gone whether or not anybody looked.

[openai_batch_error_file_audit.py](#)

<https://www.allanninal.dev/llm/batch-error-file-never-read/>

<https://github.com/allanninal/llm-api-fixes/tree/main/batch-error-file-never-read>

4 a batch expired when the 24 hour completion window closed

The submission returned 200 yesterday afternoon. The poller has been asking for the batch ever since, testing the status against `completed`, and it has never matched, so as far as the job is concerned the work is still running. It is not. Twenty-four hours after the batch started processing, everything OpenAI had not got to was abandoned, the status went to `expired`, and thirty thousand rows landed in the error file with the code `batch_expired`. Nothing raised, nothing retried, and the poller is still waiting.

[openai_batch_expiry_audit.py](#)

<https://www.allanninal.dev/llm/batch-expired-past-24h-window/>

<https://github.com/allanninal/llm-api-fixes/tree/main/batch-expired-past-24h-window>

5 a batch reads completed while some of its rows failed

The nightly job submits fifty thousand rows, sleeps, polls until the status turns to `completed`, downloads the output file and loads it. It has done that every night for eight months. The table it fills is short — not empty, not obviously wrong, just a few hundred rows smaller than the input file, by a number that changes every night. No exception was ever raised. The batch object says `completed` in plain text, and three fields further down it says `"failed": 869`, which nothing has ever read.

[openai_batch_partial_failure_audit.py](#)

<https://www.allanninal.dev/llm/batch-partial-failure-unnoticed/>

<https://github.com/allanninal/llm-api-fixes/tree/main/batch-partial-failure-unnoticed>

6 cache writes are paid for and never read back

Caching was switched on in June and the bill went up. Every call writes a fresh cache entry, is billed 1.25x base input for the privilege, and then nothing ever reads that entry back before it expires. The reason is one line: a request id got templated into the system prompt, ahead of the breakpoint, so no two prefixes have ever been byte-identical. The feature is working exactly as documented and it is costing you money.

[anthropic_cache_write_ratio.py](#)

<https://www.allanninal.dev/llm/cache-writes-with-no-reads/>

<https://github.com/allanninal/llm-api-fixes/tree/main/cache-writes-with-no-reads>

7 a floating model alias silently changes model under you

No error, no deploy, no incident. The evals were 91% on Thursday and 87% on Monday, the mean output length moved, the prompt cache hit rate dropped a few points and the bill went up slightly. Everyone looks at the prompt, which did not change, and at the retrieval corpus, which did not change either. The model changed: the config names an alias, and an alias is a pointer, not a model.

[anthropic_alias_pinning_audit.py](#)

<https://www.allanninal.dev/llm/floating-alias-instead-of-pinned-snapshot/>

<https://github.com/allanninal/llm-api-fixes/tree/main/floating-alias-instead-of-pinned-snapshot>

8 keys still work after their owner loses project access

She left in March. The laptop came back, SSO was switched off the same afternoon, and the offboarding ticket was closed with every box ticked. The API key she minted in her second week is still in the environment of a nightly job, still authenticating, still billing. Nothing revoked it, because nothing was ever asked to: the key is a separate object from her membership, and removing the membership left the key exactly as it was.

[openai_orphaned_key_audit.py](#)

<https://www.allanninal.dev/llm/key-owner-lost-project-access/>

<https://github.com/allanninal/llm-api-fixes/tree/main/key-owner-lost-project-access>

9 a model id in use is past its published shutdown date

Nothing was deployed. The key did not rotate. One model id started returning 404 on the same morning for every request, and the message reads exactly like a typo: The model does not exist or you do not have access to it. It existed yesterday. There is no distinct error code for a retired model, no deprecation warning on the successful calls that came before it, and nothing in the response that tells the difference between a model that was shut down and a model name somebody misspelled.

[openai_model_shutdown_audit.py](#)

<https://www.allanninal.dev/llm/model-past-shutdown-date/>

<https://github.com/allanninal/llm-api-fixes/tree/main/model-past-shutdown-date>

10 a model you still call retires in under 90 days

Every call is returning 200. Latency is normal, cost is normal, the evals are green. The only thing wrong is a field nobody is reading: shutdown_date on the model you route most of your traffic to is a real date, and it is close. Nothing will change until that morning, and then everything will, all at once, in every code path that names the id.

[openai_model_retirement_window.py](#)

<https://www.allanninal.dev/llm/model-retiring-within-90-days/>

<https://github.com/allanninal/llm-api-fixes/tree/main/model-retiring-within-90-days>

11 no hard spend limit is set, so the bill has no ceiling

The console shows a budget chart, and everybody who has looked at it believes it is a brake. It is not; it is a chart. The hard limit is a separate object on a separate admin endpoint, it is off by default, and until somebody turns it on there is no amount of spend in a month that will cause a request to be refused.

[openai_spend_limit_audit.py](#)

<https://www.allanninal.dev/llm/no-organization-spend-limit/>

<https://github.com/allanninal/llm-api-fixes/tree/main/no-organization-spend-limit>

12 **output tokens, not input, are what the bill is made of**

Every conversation about LLM cost starts with the prompt. The system prompt gets trimmed, the few-shot examples get cut, someone measures the context window. Then you group the cost report by token_type and find that three-quarters of the money is on the other side of the request, where none of the levers you just pulled reach.

[anthropic_output_cost_audit.py](#)

<https://www.allanninal.dev/llm/output-tokens-dominate-cost/>

<https://github.com/allanninal/llm-api-fixes/tree/main/output-tokens-dominate-cost>

13 **prompt caching was never switched on anywhere**

The system prompt is four thousand tokens of instructions, a tool catalogue and two worked examples. It is identical on every call, and there are three hundred thousand calls a month. It has been reprocessed at the full input rate every single time, because prompt caching is opt-in and nobody opted in. There is no error to find, no warning header, and no line in the invoice that says what this cost. There is only a field in the usage report that has been zero since the day the integration shipped.

[anthropic_prompt_cache_off.py](#)

<https://www.allanninal.dev/llm/prompt-caching-never-used/>

<https://github.com/allanninal/llm-api-fixes/tree/main/prompt-caching-never-used>

14 **429 credit_balance_exhausted retried forever as a rate limit**

Traffic did not degrade, it stopped. Every request comes back 429, your retry wrapper does what it was built to do, and eight hours later it is still doing it. The status code says slow down. The code field inside the body says the account is out of money, and nothing in the SDK draws a line between the two — RateLimitError is raised for both.

[openai_quota_wall_audit.py](#)

<https://www.allanninal.dev/llm/quota-exhausted-not-rate-limited/>

<https://github.com/allanninal/llm-api-fixes/tree/main/quota-exhausted-not-rate-limited>

15 **reasoning tokens are billed as output but never returned**

Somebody changed one model constant. The answers coming back are the same length as before, the prompts are the same length as before, the request count is flat — and the line for that model on the cost report went up by a factor of four. The tokens you are paying for were generated, billed at the output rate, and then not returned to you.

[openai_reasoning_token_audit.py](#)

<https://www.allanninal.dev/llm/reasoning-tokens-billed-invisibly/>

<https://github.com/allanninal/llm-api-fixes/tree/main/reasoning-tokens-billed-invisibly>

16 **a retired model id still sitting in the code**

A batch job that runs on the first of the month failed on every request with 404 and "type": "not_found_error", message The requested resource could not be found. The endpoint is right, the key works, the same key runs the rest of the application all day. The model id in that job's params block was retired months ago, and nothing else in the codebase names it, so nothing else broke and nobody knew.

[anthropic_model_ids_audit.py](#)

<https://www.allanninal.dev/llm/retired-model-id-still-in-code/>

<https://github.com/allanninal/llm-api-fixes/tree/main/retired-model-id-still-in-code>
